

## Aula 4 — PWA, Service Workers & Offline-First

*"PWA é só web glorificada?"*

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 24/06/2026

## Recap da Aula 3

- Comparativo aprofundado RN, Flutter, KMP
- Native module bridging com TurboModule
- Estudos de caso Airbnb, Discord, Coinbase

**Atividade 3** — entregue?

## Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Service Workers, lifecycle, cache strategies
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Hands-on: converter SPA em PWA com offline-first
4. **Esquenta (15min)** — Atividade 4

## Objetivos de aprendizagem

- **Compreender** ciclo de vida de Service Workers (install, activate, fetch)
- **Aplicar** cache strategies (Cache-first, Network-first, Stale-While-Revalidate)
- **Implementar** offline-first com IndexedDB + Background Sync
- **Avaliar** PWA vs RN-Web vs Capacitor para casos específicos
- **Configurar** Lighthouse CI bloqueando PRs com regressão

## "PWA é só web?" — fato vs ficção

**Twitter Lite:** ~30% do tráfego mobile do Twitter já em PWA, com offline real.

**Starbucks:** PWA com pedido offline robusto.

**Pinterest Lite:** instalável, push, offline.

**Spotify Web:** virou PWA instalável em 2023.

**PWA não é cosmético.** Em 2026, com Background Sync + Web Push + offline + install desktop, cobre casos antes exclusivos de nativo.

## Anatomia de uma PWA

Uma PWA precisa de **3 componentes**:

1. **Web App Manifest** ( `manifest.json` ) — metadados (nome, ícones, cores, display)
2. **Service Worker** — interceptador de network, cache, offline
3. **HTTPS** — origem segura obrigatória

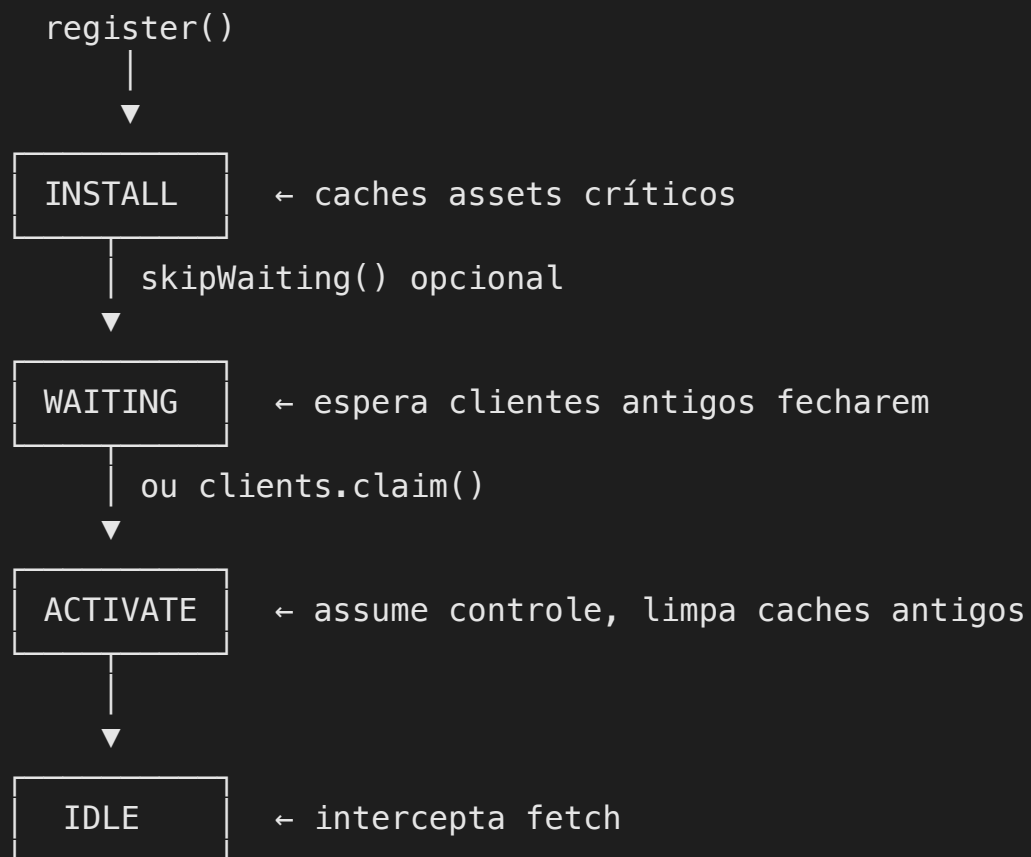
Bonus para "instalável":

- Ícones em múltiplos tamanhos
- `start_url`, `display: standalone`
- Service Worker com `fetch handler`

## Web App Manifest

```
{
  "name": "Minha PWA",
  "short_name": "MinhaApp",
  "description": "Descrição curta da app",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#003366",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ]
}
```

## Service Worker — lifecycle





## Service Worker — install + activate

```
// sw.js
const CACHE_NAME = 'app-v3';
const PRECACHE_URLS = ['/', '/index.html', '/styles.css', '/app.js'];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => cache.addAll(PRECACHE_URLS))
  );
  self.skipWaiting(); // assume controle imediato (cuidado!)
});

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((names) =>
      Promise.all(
        names.filter((n) => n !== CACHE_NAME).map((n) => caches.delete(n))
      )
    )
  );
  self.clients.claim();
});
```

## Cache strategies

### Cache-first (estáticos)

Tenta cache; se falhar, vai pra rede; cacheia o resultado.

### Network-first (dados dinâmicos)

Tenta rede com timeout; se falhar, vai pro cache.

### Stale-While-Revalidate

Serve cache imediatamente (rápido) + busca rede em paralelo (atualiza próxima request).

### Cache-only / Network-only

Casos específicos (assets imutáveis vs telemetria).

## Cache-first — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/static/')) {  
    event.respondWith(  
      caches.match(event.request).then((cached) => {  
        return cached || fetch(event.request).then((response) => {  
          const clone = response.clone();  
          caches.open(CACHE_NAME).then((cache) => cache.put(event.request, clone));  
          return response;  
        });  
      });  
    );  
  }  
});
```

Use para JS, CSS, imagens, fonts versionadas.

## Stale-While-Revalidate — implementação

```
self.addEventListener('fetch', (event) => {  
  if (event.request.url.includes('/api/feed')) {  
    event.respondWith(  
      caches.open(CACHE_NAME).then(async (cache) => {  
        const cached = await cache.match(event.request);  
        const fetchPromise = fetch(event.request).then((response) => {  
          cache.put(event.request, response.clone());  
          return response;  
        });  
        return cached || fetchPromise;  
      })  
    );  
  }  
});
```

Usuário vê algo imediatamente, próxima carga já está atualizada.

## Workbox — abstração madura

```
import { precacheAndRoute } from 'workbox-precaching';
import { registerRoute } from 'workbox-routing';
import {
  CacheFirst,
  NetworkFirst,
  StaleWhileRevalidate,
} from 'workbox-strategies';

precacheAndRoute(self.__WB_MANIFEST);

registerRoute(/\.(?:png|jpg|svg)$/ , new CacheFirst({ cacheName: 'images' }));
registerRoute(
  /\api\/static\/ ,
  new StaleWhileRevalidate({ cacheName: 'api-static' })
);
registerRoute(/\api\/dynamic\/ , new NetworkFirst({ cacheName: 'api-dynamic' }));
```

Workbox elimina boilerplate. Use em produção.

## IndexedDB — persistência local estruturada

`localStorage` é síncrono e limitado (~5MB). **IndexedDB** é a opção real.

```
import Dexie from 'dexie';

const db = new Dexie('MyAppDB');
db.version(1).stores({
  drafts: '++id, content, createdAt',
  user: '&id, name, email',
});

// Add
await db.drafts.add({ content: 'Hello', createdAt: Date.now() });

// Query
const recent = await db.drafts
  .orderBy('createdAt')
  .reverse()
  .limit(10)
  .toArray();
```

Dexie torna IndexedDB usável.

## Background Sync API

Usuário fecha app sem conexão. Operação fica pendente. Quando conectividade volta, **Service Worker dispara sync event mesmo com app fechado.**

```
// Page side
async function submitOfflineSafe(message) {
  await db.outbox.add({ message, attempts: 0 });

  if ('serviceWorker' in navigator && 'SyncManager' in window) {
    const registration = await navigator.serviceWorker.ready;
    await registration.sync.register('flush-outbox');
  }
}

// sw.js
self.addEventListener('sync', (event) => {
  if (event.tag === 'flush-outbox') {
    event.waitUntil(flushOutbox());
  }
});
```

## Web Workers vs Service Workers

| Aspecto              | Web Worker                | Service Worker              |
|----------------------|---------------------------|-----------------------------|
| Propósito            | Compute pesado off-thread | Network proxy + cache       |
| Lifecycle            | Scope da página           | Persistente, instalado      |
| Escopo               | Página atual              | Origin/scope inteiro        |
| API                  | postMessage com page      | fetch , cache , push , sync |
| Múltiplas instâncias | 1 por chamada             | 1 por scope                 |

Use Web Worker para parsing JSON gigante, image processing, ML em browser.



## Lighthouse CI

```
# .github/workflows/lighthouse.yml
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
```

```
// lighthouseci.json
{
  "ci": {
    "assert": {
      "assertions": {
        "categories:performance": ["error", { "minScore": 0.9 }],
        "categories:pwa": ["error", { "minScore": 0.9 }]
      }
    }
  }
}
```

## PWA vs RN-Web vs Capacitor

| Aspecto           | PWA            | RN-Web             | Capacitor         |
|-------------------|----------------|--------------------|-------------------|
| Distribuição      | URL ou install | URL ou install     | App Store + Play  |
| Capacidade nativa | Limitada       | Limitada           | Total via plugins |
| Tamanho bundle    | Pequeno        | Médio              | Maior             |
| Performance       | Web            | Web (com RN-style) | Nativo wrapped    |
| Update            | Instantâneo    | Instantâneo        | Via store         |

Cada um cobre nichos. Não competem diretamente.

## Intervalo — 30min

Volta às :\_\_\_\_

Aproveita pra:

- Esticar perna
- Abrir Chrome DevTools (Application tab) — vamos usar muito
- Tomar café / água

Próximo bloco é converter SPA em PWA. DevTools + projeto base prontos = ganha 5min.

## Hands-on — vamos fazer juntos (75min)

### Roteiro

1. Clone starter `aula-04` — SPA React + Vite (5min)
2. Adicionar `manifest.json` (10min)
3. Configurar Workbox (15min)
4. Implementar SW com cache híbrido (20min)
5. Persistência offline com Dexie (15min)
6. Background Sync de uma operação write (15min)
7. Lighthouse CI bloqueando (10min)

## Pitfalls comuns

- ✗ `skipWaiting()` sem cuidado — clientes abertos podem ficar inconsistentes
- ✗ **Cache infinito** — esquecer de limpar caches antigos no `activate`
- ✗ `localStorage` ao invés de `IndexedDB` — limite de 5MB e síncrono
- ✗ **Service Worker em scope errado** — não intercepta o que deveria
- ✗ **Não testar em "modo airplane"** — só descobre bug em produção

## Atividade 4: PWA Offline-First (15 pts)

**Entrega: até 23/06.**

Entregar PWA com:

1. **Lighthouse PWA score  $\geq 90$**  (screenshot do CI obrigatório)
2. **SW com cache strategy híbrida** justificada por endpoint
3. **IndexedDB** com persistência e migration
4. **Background Sync** de pelo menos 1 operação write
5. **Relatório 2pgs** explicando trade-offs da estratégia

**Critérios detalhados de avaliação:** Canvas (módulo Atividade 4).

## Leituras obrigatórias (pré-Aula 5)

- **Ater, T.** — *Building Progressive Web Apps* (O'Reilly)
- **Archibald, J.** — *The Offline Cookbook* (web.dev)
- **W3C** — *Service Worker Specification* (parte 1 e 2)
- **Newman, S.** — *Building Microservices* 2ed, capítulo 4 (BFF)
- **Porcello & Banks** — *Learning GraphQL* (O'Reilly)

## Próxima aula (24/06)

### Aula 5 — BFF, GraphQL, Segurança

Pré-requisito:

- Atividade 4 entregue (23/06)
- Apollo Server local rodando (ou Hasura)
- Conta Auth0 ou Cognito (free tier)



# Obrigado!

 [jackson.96@gmail.com](mailto:jackson.96@gmail.com)

 [linkedin.com/in/3jacksonsmith](https://linkedin.com/in/3jacksonsmith)

 [github.com/jacksonsmith](https://github.com/jacksonsmith)

**Próxima quarta, 24/06, 19:00**

## KotlinProject — como foi criado

Projeto gerado pelo **KMP wizard oficial** da JetBrains:

[kmp.jetbrains.com](https://kmp.jetbrains.com)

Config usada: Android ✓ · iOS ✓ · iOS UI = **Compose** · Include Tests ✓

Essa URL reproduz exatamente o starter. Qualquer aluno pode gerar o mesmo projeto do zero no wizard — é o ponto de partida oficial pra projetos KMP novos.

## O que vamos rodar hoje — KotlinProject

Três apps, um módulo compartilhado ( `shared` — Compose Multiplatform):

| App                     | Módulo                   | Plataforma          |
|-------------------------|--------------------------|---------------------|
| <code>androidApp</code> | <code>:androidApp</code> | Android             |
| <code>iosApp</code>     | <code>iosApp/</code>     | iOS (Xcode project) |
| <code>webApp</code>     | <code>:webApp</code>     | Web (JS / Wasm)     |

Targets declarados em `shared/build.gradle.kts`:

`androidTarget`, `iosArm64`, `iosSimulatorArm64`, `js`, `wasmJs`

## Toolchain verificado — versões exatas

| Ferramenta            | Versão                  |
|-----------------------|-------------------------|
| Android Studio        | Quail 2026.1.1 Patch 2  |
| Gradle wrapper        | 9.1.0                   |
| Kotlin                | 2.4.0                   |
| AGP                   | 9.0.1                   |
| Compose Multiplatform | 1.11.1                  |
| JDK (terminal)        | OpenJDK 17.0.15         |
| JDK (Studio)          | bundled OpenJDK 21.0.10 |
| Xcode                 | 26.1.1                  |

Projeto **não carrega** no Studio < Quail 2026.1.1. Atualizar se necessário.

## Setup — pré-requisitos

```
# 1. Verificar Xcode CLI
xcode-select -p
# → /Applications/Xcode.app/Contents/Developer ✓
```

- **macOS** + Homebrew ( `/opt/homebrew/bin/brew` )
- **Android Studio Quail 2026.1.1+** — versão exata importa
- **Xcode 26.x** com command-line tools selecionados
- `local.properties` criado automaticamente pelo Studio:

```
sdk.dir=/Users/<seu-user>/Library/Android/sdk
```

| Não commitar — é caminho de máquina.

## Setup — plugin KMP no Android Studio

A parte mais traiçoeira. A configuração `iosApp` só aparece depois que o plugin carrega.

### Passo a passo:

1. **Settings → Plugins → Marketplace** → buscar **Kotlin Multiplatform** (JetBrains s.r.o.) → Install
2. Se aparecer erro **Requires plugin 'com.intellij.marketplace' to be installed** + badge vermelho:
  - Marketplace → buscar **JetBrains Marketplace Licensing** → Install
3. Reiniciar o Studio
4. Dropdown de run config agora mostra `iosApp` ao lado de `androidApp` ✓

`kdoctor` pode reportar "KMM plugin not installed" — é falso positivo (procura o id legado 14936). Ignorar.

## Setup — iOS Simulator (se não existir)

```
# Checar devices disponíveis
xcrun simctl list devices available

# Se a lista estiver vazia sob um runtime, criar:
xcrun simctl create "iPhone 16" \
    com.apple.CoreSimulator.SimDeviceType.iPhone-16 \
    com.apple.CoreSimulator.SimRuntime.iOS-26-1
```

Sintoma sem device: kdoctor avisa *"Couldn't find available Apple Simulators"*.

## Verificar ambiente — kdoctor

```
brew install kdoctor  
kdoctor # ou /opt/homebrew/bin/kdoctor
```

**Esperado:** OS ✓ · Java ✓ · Xcode ✓

**2 warnings seguros para ignorar:**

- *"Android Studio: KMM plugin not installed"* — kdoctor 1.1.0 checa id legado; nosso plugin está ok
- *"CocoaPods not configured"* — este projeto não tem `Podfile` ; só necessário com libs CocoaPods



## Rodando os 3 targets

### Android

- Dropdown → `androidApp` + emulador/device → 

### iOS

- Dropdown → `iosApp` + `iPhone 16` → 
- Fallback (sem plugin):

```
open iosApp/iosApp.xcodeproj    # Run pelo Xcode
```

### Web

```
./gradlew :webApp:jsBrowserDevelopmentRun    # JS  
./gradlew :webApp:wasmJsBrowserDevelopmentRun    # Wasm
```

## Gotchas do KotlinProject

- **Apple Silicon:** target é `iosSimulatorArm64`. `iosX64` **não** é declarado → simuladores Intel não buildam.
- **Config cache com comportamento estranho:**

```
./gradlew --no-configuration-cache <task>
```

- **Nunca commitar** `local.properties` — caminho do SDK é específico de máquina.

## Kotlin Multiplatform (KMP) — Stack & Forças

### Stack:

- **Kotlin** linguagem em Android, iOS via Kotlin/Native, JS, JVM, Linux, Windows
- **expect / actual** — contratos compartilhados, implementações por plataforma
- **Compose Multiplatform** — UI compartilhada (em maturação)
- **Ktor** — HTTP client
- **kotlinx.serialization** — JSON

### Forças:

- Compartilha **regra de negócio**, não UI (na maioria dos casos)
- Excelente integração com Android nativo
- iOS via Swift interop

## Kotlin Multiplatform (KMP) — Fraquezas

- Talent pool ainda restrito
- iOS interop com debugging chato
- Compose Multiplatform ainda jovem em iOS

## Kotlin — sintaxe em 1 slide

```
val nome = "Ana"           // imutável (≈ const)
var contador = 0           // mutável
fun saudar(n: String) = println("Oi, $n")    // função

data class Filme(val titulo: String, val nota: Double) // equals/copy/toString grátis

val apelido: String? = null // ? = pode ser null
val tam = apelido?.length ?: 0 // safe-call (?.) + elvis (?:)
listOf(1, 2, 3).filter { it > 1 } // lambda: it = o item
```

`val / var` · `data class` = seu modelo · `? / ?:` = null-safety · `{ it }` = lambda. É a linguagem do "shared" do KMP.

## KMP — arquitetura (shared + targets)

**commonMain** — regra de negócio compartilhada

models, repositórios, casos de uso · `expect` declara o que muda por plataforma

▼ compila pra cada target ▼

**androidMain**

`actual` → OkHttp, APIs Android

**iosMain**

`actual` → URLSession, APIs iOS

Compartilha **lógica**, não UI. O que é específico de plataforma fica nos `actual`.

## KMP — `expect` / `actual`

```
// commonMain
expect class HttpClient {
    suspend fun get(url: String): String
}

// androidMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // OkHttp implementation
    }
}

// iosMain
actual class HttpClient {
    actual suspend fun get(url: String): String {
        // NSURLSession implementation
    }
}
```

`expect` declara o contrato. `actual` implementa por plataforma.

## Hands-on guiado — Kotlin (Playground)

[play.kotlinlang.org](https://play.kotlinlang.org) (zero instalação). Escreva a "shared logic" dos favoritos — a mesma do `favoritesStore` do app:

```
data class Movie(val id: Int, val title: String, val rating: Double) // 1. modelo

fun favorites(movies: List<Movie>, favIds: Set<Int>) = // 2. função pura
    movies.filter { it.id in favIds }

fun main() {
    val all = listOf(
        Movie(1, "Matrix", 8.7),
        Movie(2, "Inception", 8.8),
        Movie(3, "Tenet", 7.4),
    )
    val favs = favorites(all, setOf(1, 3)) // 3. rode!
    println(favs.map { it.title }) // → [Matrix, Tenet]
}
```

(1) `data class` = modelo · (2) função pura de filtro · (3) rode. Essa lógica o KMP compartilha entre Android e iOS — só o que toca plataforma (rede, sensores) vira `expect / actual`.



## Estudos de caso — KMP

### KMP:

- Square (Cash App)
- JetBrains (próprio Toolbox, Fleet)
- McDonald's mobile
- Philips

## BFF — Backend-for-Frontend

### O problema sem BFF

- RN, KMP, Web cada um bate em endpoints REST diferentes
- Over-fetching (campos que não usa) e under-fetching (N+1 requests)
- Múltiplos times sincronizando formatos de resposta

### Com BFF + GraphQL

- Um endpoint `/graphql` serve **todos os clientes**
- Cliente pede **só os campos que precisa**
- Schema = contrato único versionado

```
# Android pergunta isso:
query {
  popularMovies {
    results { title voteAverage }
  }
}

# PWA pergunta isso:
query {
  popularMovies {
    results { title posterPath overview }
  }
}
```

**Mesma query shape, respostas diferentes.** O BFF otimiza por cliente sem mudar o backend.

## GraphQL na prática — o servidor CineHub

```
# No terminal — pasta compartilhado/capstone/server  
npm install && npm run seed && npm start  
# → CineHub GraphQL pronto em http://localhost:4000/
```

Abra <http://localhost:4000/> no browser — Playground Apollo Studio.

Queries disponíveis:

```
{ popularMovies(page: 1) { page results { id title voteAverage } } }  
  
{ searchMovies(query: "Matrix") { id title releaseDate } }  
  
{ movie(id: 550) { title overview } }
```

O CineHub tem dados de filmes clássicos no SQLite local. Sem TMDB token necessário — ótimo pra demo.

## TMDB REST vs GraphQL — dois sabores do mesmo problema

|                | TMDB REST (exercícios)    | CineHub GraphQL (demo)            |
|----------------|---------------------------|-----------------------------------|
| Autenticação   | Bearer token no header    | JWT local (register/login)        |
| Popular movies | GET /movie/popular        | query { popularMovies }           |
| Busca          | GET /search/movie?query=x | query { searchMovies(query:"x") } |
| Over-fetching  | Sim (resposta fixa)       | Não (cliente escolhe campos)      |
| Offline        | Cache HTTP / SW           | Normalização client-side          |

**No KMP** usamos Ktor + TMDB REST — simples, sem servidor local.

**Na PWA** usamos fetch + TMDB REST + Service Worker para offline.

**No BFF real** (capstone) tudo passa pelo GraphQL.

## Atividade 6 — KMP: filmes populares (15 pts)

**Stack:** KotlinProject (Android) + Ktor + TMDB REST API

O que você vai implementar no `shared/src/commonMain/` :

```
data/  
  Movie.kt      ← @Serializable (já pronto)  
  TmdbApi.kt    ← TODO 1: implementar popularMovies() com Ktor  
  App.kt        ← TODO 2: LaunchedEffect + TODO 3: LazyColumn
```

**Setup:**

```
cd exercicios/06-kmp/pratica  
cp local.properties.example local.properties  
# adicione seu tmdb.token= no local.properties  
# Sync Gradle → Run androidApp
```

**Entrega:** fork → PR `feat(a6-kmp): <login>` · Prazo: canvas

## Atividade 7 — PWA: filmes instalável + offline (15 pts)

**Stack:** React + Vite + vite-plugin-pwa (Workbox) + TMDB REST

O que você vai implementar:

```
src/hooks/useTmdb.ts    ← TODO 1: fetchPopularMovies() com fetch()
                        ← TODO 2: useEffect + loading/error
vite.config.ts          ← TODO 3: runtimeCaching NetworkFirst pra TMDB
```

Setup:

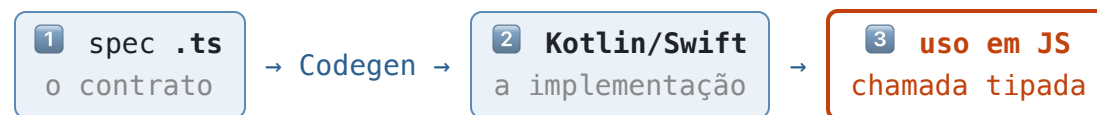
```
cd exercicios/07-pwa-tmdb/pratica
npm install
cp .env.example .env.local    # adicione VITE_TMDB_TOKEN
npm run dev                   # http://localhost:5173
```

Teste offline:

```
npm run build && npm run preview
# DevTools → Application → Service Workers → "Offline" → F5
```

## [MOVIDO da Aula 3] Native bridging

### Native bridging RN ↔ Kotlin (Android)



Os 3 lugares de todo native module — é só preencher cada um.

#### Passo 1: definir spec TS

```
// specs/NativeBiometrics.ts
import { TurboModule, TurboModuleRegistry } from 'react-native';

export interface Spec extends TurboModule {
  isAvailable(): Promise<boolean>;
  authenticate(reason: string): Promise<boolean>;
}

export default TurboModuleRegistry.getEnforcing<Spec>('NativeBiometrics');
```

#### Passo 2: rodar Codegen

## Native bridging — Kotlin

```
// android/src/main/java/com/myapp/NativeBiometricsModule.kt
package com.myapp

import com.facebook.react.bridge.*
import androidx.biometric.BiometricPrompt

class NativeBiometricsModule(reactContext: ReactApplicationContext)
    : NativeBiometricsSpec(reactContext) {

    override fun isAvailable(promise: Promise) {
        val biometricManager = BiometricManager.from(reactApplicationContext)
        val canAuth = biometricManager.canAuthenticate(
            BiometricManager.Authenticators.BIOMETRIC_STRONG
        ) == BiometricManager.BIOMETRIC_SUCCESS
        promise.resolve(canAuth)
    }

    override fun authenticate(reason: String, promise: Promise) {
        // BiometricPrompt setup...
        promise.resolve(true)
    }

    override fun getName() = "NativeBiometrics"
}
```



## Native bridging — Swift

```
// ios/NativeBiometrics.swift
import Foundation
import LocalAuthentication

@objc(NativeBiometrics)
class NativeBiometrics: NSObject {

    @objc
    func isAvailable(_ resolve: @escaping RCTPromiseResolveBlock,
                    rejecter reject: @escaping RCTPromiseRejectBlock) {
        let context = LAContext()
        var error: NSError?
        let canEval = context.canEvaluatePolicy(
            .deviceOwnerAuthenticationWithBiometrics,
            error: &error
        )
        resolve(canEval)
    }

    @objc
    func authenticate(_ reason: String,
                     resolver resolve: @escaping RCTPromiseResolveBlock,
                     rejecter reject: @escaping RCTPromiseRejectBlock) {
        let context = LAContext()
        context.evaluatePolicy(
            .deviceOwnerAuthenticationWithBiometrics,
            localizedReason: reason
        ) { success, error in
            if let error = error {
                reject("auth_failed", error.localizedDescription, error)
                return
            }
            resolve(success)
        }
    }
}
```

## Native bridging — uso em RN

```
import NativeBiometrics from './specs/NativeBiometrics';

async function login() {
  const available = await NativeBiometrics.isAvailable();

  if (!available) {
    Alert.alert('Biometria não disponível neste device');
    return;
  }

  const ok = await NativeBiometrics.authenticate(
    'Confirme sua identidade para continuar'
  );

  if (ok) {
    navigation.navigate('Dashboard');
  }
}
```

Mesma API JS, implementações nativas distintas. **Type-safe** ponta a ponta.

 [reactnative.dev/docs/turbo-native-modules-introduction](https://reactnative.dev/docs/turbo-native-modules-introduction) · [docs.expo.dev/modules](https://docs.expo.dev/modules)

## Native bridging — instalar + exercício GPS (75min)

⚠ **Mudamos de stack:** Flutter foi a **comparação** (Bloco 1). Agora voltamos pro **React Native** — o app da Atividade 2 — pra ver como o **RN fala com o nativo**. O app é RN → usa **npm**; a ponte é **JS/TS** ↔ **Kotlin**.

Agora a gente **começa a instalação** (Android Studio baixa em paralelo) e quem subir o ambiente faz **1 exercício GPS guiado**. Quem travar acompanha a demo e termina em casa ( `#aula-03-setup` ).

### Roteiro

1. **Kickoff da instalação** (paralelo): Android Studio + JDK 17 + emulador + `npm install` no `pratica/` — deixa baixando (20min)
2. **Conceito + demo** enquanto baixa: os **3 lugares** (tipo TS → Kotlin → uso); starter rodando ( `getBatteryLevel` ) (15min)
3. **Exercício GPS guiado:** implementar `getCurrentLocation()` e testar com **mock location no emulador** (30min)
4. Recap + comparativo: RN (TurboModule) × Flutter (Platform Channel) × Nativo (10min)

O `getCurrentLocation` de hoje é a **Parte 1 da Atividade 3 começando**. Em casa: + `isLocationEnabled`, usar na tela + relatório (prazo 23/06).

## Pitfalls comuns no bridging

- ✗ Esquecer de habilitar **New Architecture** — TurboModule não funciona
- ✗ Não rodar **Codegen** após mudar spec — bindings desatualizados
- ✗ **Bridge** legado em vez de **TurboModule** — perde performance e tipo
- ✗ **Promise** sem **reject** em erro nativo — JS pendura forever
- ✗ Acesso a UI thread iOS sem `DispatchQueue.main` — crash silencioso